Ingegneria del Software: Esercitazione Java – Incapsulamento

Molka Gharbaoui

Caratteristiche generali

Caratteristiche generali

- Il paradigma dell'incapsulamento consente di creare programmi composti da classi, che nascondono dettagli di implementazione dietro interfacce pubbliche.
- Questi dettagli non sono necessari per usare le istanze create da quella classe.
- L'incapsulamento favorisce il riuso del codice già scritto e semplifica il lavoro del programmatore.

Caratteristiche generali

- L’incapsulamento separa la definizione di una classe in due parti:
  - L’interfaccia: indica ai programmatori ciò di cui hanno bisogno per usare la classe nei loro programmi;
  - L’implementazione: consiste in tutti gli elementi privati della classe, principalmente le variabili di istanza private e le definizioni dei metodi pubblici e privati.
- Dall’esterno si sa cosa fa un oggetto, ma non come lo fa.

Esempi di codice

Esempi di codice

- Esempio 1: Definire una semplice classe Number che permette di visualizzare un numero intero. Mostrare l’uso dell’incapsulamento nel codice.

Esempi di codice

Number.java

Source History

```
1   public class Number
2   {
3      private int y;
4
5   public int get() {
6       return y;
7   }
8   public void set(int y) {
9      this.y = y;
10   }
11  }
```

Esempi di codice

Number.java | Encaptest.java

Source | History

```
1   public class Encaptest
2   {
3   public static void main(String[] args)
4   {
5    Number n = new Number();
6      n.set( y: 6);
7
8     int num = n.get();
9     System.out.println( x: num);
10    }
11  }
12
```

Esempi di codice

- Esempio 2: Astrarre il concetto di Moneta con una classe assumendo che tutte le monete avranno come valuta l'EURO, ed avranno la variabile d'istanza incapsulata valore. Creare un vincolo affinché le monete possano essere istanziate obbligatoriamente con un valore.

Esempi di codice

Moneta.java

Source | History

```java
public class Moneta {

    public final static String VALUTA = "EURO";

    private int valore;

    public Moneta(int valore) {
        this.valore = valore;
        System.out.println("Creata una moneta da " + valore + " centesimi");
    }

    public void setValore(int valore) {
        this.valore = valore;
    }

    public int getValore() {
        return valore;
    }
}
```

Esempi di codice

- Esempio 3: Creare una classe Utente che identifica un utente con username e password usando il paradigma d’incapsulamento.

Esempi di codice

Utente.java

Source | History

```java
public class Utente {
    private String nome;
    private String username;
    private String password;

    public Utente(String n, String u, String p) {
        this.nome = n;
        this.username = u;
        this.password = p;}

    public String getNome() {
        return nome;}

        public String getUsername() {
        return username;}

    public String getPassword() {
        return password;}

    public void setNome(String nome) {
        this.nome = nome;}

    public void setUsername(String username) {
        this.username = username;}

    public void setPassword(String password) {
        this.password = password;}
}
```

Esempi di codice

- Esempio 4: Creare una classe Libro incapsulata, in modo tale che astragga il concetto di un libro che può essere venduto in una libreria. Tra i campi definiti dalla classe Libro, deve esserci il campo genere litterario. Fare in modo che un libro possa essere associato solo ad un genere letterario compreso tra un insieme di generi letterari predefiniti, per esempio l'insieme costituito dai generi romanzo, thriller, saggio, manuale.

Esempi di codice

Libro.java

Source | History

```java
public class Libro {

    private String titolo;
    private String autore;
    private int prezzo;
    private String genere;

    public static final String [] generi = {"Romanzo", "Saggio", "Thriller","Manuale"};

    public Libro(String titolo, String autore, int prezzo, String genere) {
        setTitolo(titolo);
        setAutore(autore);
        setPrezzo(prezzo);
        setGenere(genere);}

    public String getTitolo() {
        return titolo;}

    public String getAutore() {
        return autore;}

        public int getPrezzo() {
        return prezzo;}

    public String getGenere() {
        return genere;
    }
```

Esempi di codice

```
29      private void setTitolo(String titolo) {
30          this.titolo = titolo;}
31
32      private void setAutore(String autore) {
33          this.autore = autore;}
34
35      private void setPrezzo(int prezzo) {
36          this.prezzo = prezzo;
37      }
38
39      private void setGenere(String genere) {
40          for (String generePredefinito : generi) {
41              if (generePredefinito.equals( anObject: genere)) {
42                  this.genere = genere;
43                  return;
44              }
45          }
46          System.out.println("Genere " + genere + " non valido! Usare uno dei seguenti generi:");
47          for (String generePredefinito : generi) {
48              System.out.println( x: generePredefinito);
49          }
50      }
51  }
52
```

Esempi di codice

- Esempio 5: Creare una classe Frazione per rappresentare le frazioni intere e le operazioni su di esse. Nella classe, una frazione viene sempre ridotta ai minimi termini e con numeratore positivo.
- Una frazione viene definita come una espressione (num/den) con den>0.

Esempi di codice

Frazione.java

Source | History

```
1
2   public class Frazione {
3
4       private int num, den;
5
6       public Frazione(int num, int den)  {
7           assert den!=0: "denominatore frazione deve essere diverso da 0";
8           this.num=num; this.den=den; }
9
10      public void setFrazione(int n, int d)  {
11          assert den!=0: "denominatore frazione deve essere diverso da 0";
12          this.num=n; this.den=d; }
13
14      public boolean equals(Frazione f)
15          {return (this.num * f.den == this.den * f.num);}
16
17      public Frazione somma(Frazione f) {
18
19              int n= this.num*f.den + this.den * f.num; int d= this.den*f.den;
20              return new Frazione( num: n, den: d);}
21
22      public Frazione prodotto(Frazione f){
23
24              int n= this.num*f.num; int d= this.den*f.den;
25              return new Frazione( num: n, den: d);}}
```